# PROJET

Gestion des étudiants et des résultats académiques

Programmation orientée objet : modélisation UML et implémentation C++

**NOUHOUMON K. Lucien Sèjlo**
Matricule : **11061STI25**
Licence 3 Mathématiques et Informatique
FAST-Natitingou – UNSTIM

**Hypothèses retenues :** notes sur 20, seuil de validation 10/20, et un *coefficient* par matière (nécessaire pour calculer une moyenne pondérée).

## 1 Identification des classes

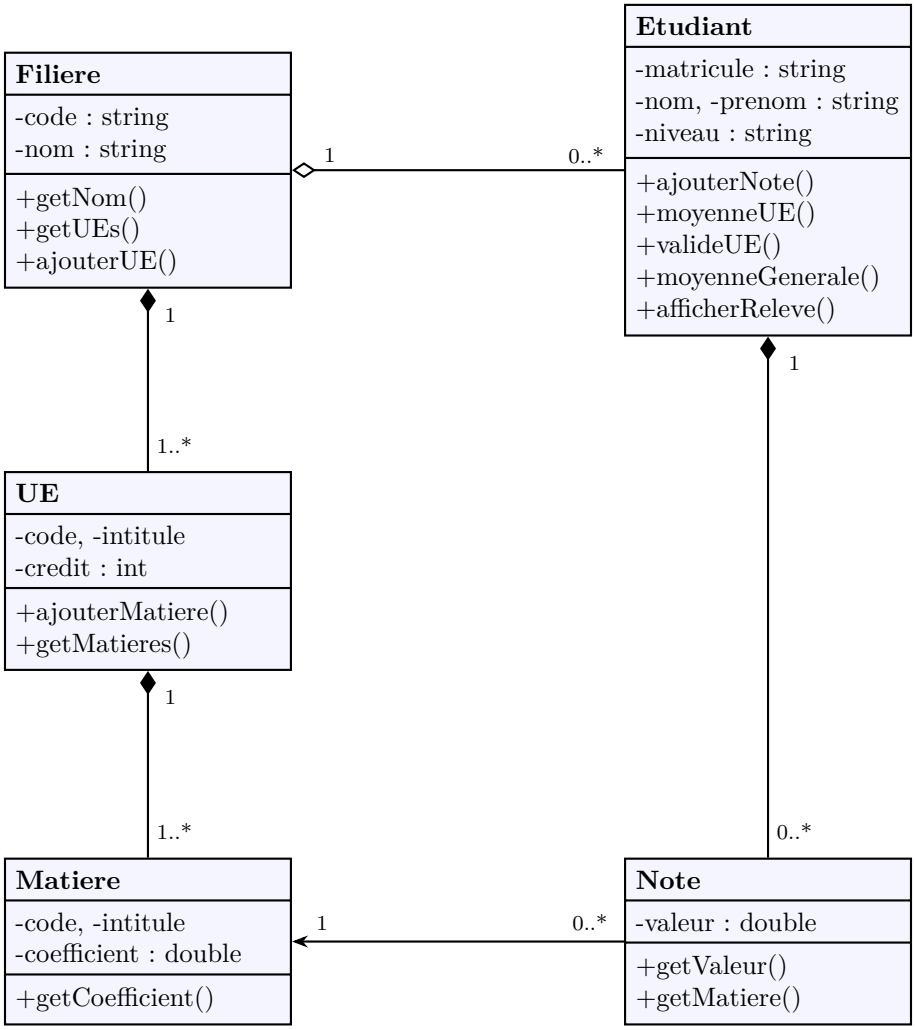
| Classe | Attributs | Rôle |
|---|---|---|
| `Etudiant` | matricule, nom, prénom, niveau (L1/L2/L3) | personne évaluée |
| `Filiere` | code, nom | regroupe les UE |
| `UE` | code, intitulé, crédit | regroupe les matières |
| `Matiere` | code, intitulé, coefficient | enseignement noté |
| `Note` | valeur | résultat d'un étudiant dans une matière |

`Note` est une **classe d'association** entre `Etudiant` et `Matiere` : une note n'existe que pour un couple (étudiant, matière).

## 2 Relations et multiplicités

| Relation | Type | Mult. | Justification |
|---|---|---|---|
| Filiere → Etudiant | **Agrégation** ◇ | 1 – 0..∗ | un étudiant existe indépendamment de sa filière |
| Filiere → UE | **Composition** ◆ | 1 – 1..∗ | une UE n'a pas de sens sans sa filière |
| UE → Matiere | **Composition** ◆ | 1 – 1..∗ | une matière appartient à une seule UE |
| Etudiant → Note | **Composition** ◆ | 1 – 0..∗ | les notes disparaissent avec l'étudiant |
| Note → Matiere | **Association** | 0..∗ – 1 | une note concerne une seule matière |

# 3 Diagramme de classes UML




*Légende : ♦ composition, ◊ agrégation, → association orientée.*

# 4 Formules de calcul

**Moyenne d'une UE** (moyenne pondérée par les coefficients des matières) :

$$M_{UE} = \frac{\sum_i n_i \times c_i}{\sum_i c_i}$$

**Validation d'une UE :**

$$\text{UE validée} \iff M_{UE} \geq 10$$

**Moyenne générale** (pondérée par les crédits) :

$$M_G = \frac{\sum_j M_{UE_j} \times \text{crédit}_j}{\sum_j \text{crédit}_j}$$

# 5 Implémentation C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <stdexcept>
using namespace std;

// ---------- Matiere ----------
class Matiere {
    string code, intitule;
    double coefficient;
public:
    Matiere(string c, string i, double coef)
        : code(c), intitule(i), coefficient(coef) {
        if (coef <= 0) throw invalid_argument("Coefficient invalide");
    }
    string getCode() const { return code; }
    string getIntitule() const { return intitule; }
    double getCoefficient() const { return coefficient; }
};

// ---------- UE (composition de Matiere) ----------
class UE {
    string code, intitule;
    int credit;
    vector<Matiere> matieres;           // composition
public:
    UE(string c, string i, int cr) : code(c), intitule(i), credit(cr) {}
    void ajouterMatiere(const Matiere& m) { matieres.push_back(m); }
    string getCode() const { return code; }
    string getIntitule() const { return intitule; }
    int getCredit() const { return credit; }
    const vector<Matiere>& getMatieres() const { return matieres; }
};

// ---------- Filiere (composition d'UE) ----------
class Filiere {
    string code, nom;
    vector<UE> ues;                     // composition
public:
    Filiere(string c, string n) : code(c), nom(n) {}
    void ajouterUE(const UE& ue) { ues.push_back(ue); }
    string getNom() const { return nom; }
    const vector<UE>& getUEs() const { return ues; }
};

// ---------- Note (classe d'association) ----------
class Note {
    const Matiere* matiere;             // association
    double valeur;
public:
    Note(const Matiere* m, double v) : matiere(m), valeur(v) {
        if (v < 0 || v > 20) throw invalid_argument("Note hors [0;20]");
    }
    const Matiere* getMatiere() const { return matiere; }
    double getValeur() const { return valeur; }
};

// ---------- Etudiant ----------
class Etudiant {
    string matricule, nom, prenom, niveau;
    const Filiere* filiere;             // agregation
```

```
    vector<Note> notes;                 // composition
public:
    Etudiant(string mat, string n, string p, string niv, const Filiere* f)
        : matricule(mat), nom(n), prenom(p), niveau(niv), filiere(f) {}

    void ajouterNote(const Matiere& m, double valeur) {
        notes.emplace_back(&m, valeur);
    }

    // Note dans une matiere (0 si non encore saisie)
    double noteDe(const Matiere& m) const {
        for (const Note& n : notes)
            if (n.getMatiere()->getCode() == m.getCode())
                return n.getValeur();
        return 0.0;
    }

    double moyenneUE(const UE& ue) const {
        double somme = 0, coefs = 0;
        for (const Matiere& m : ue.getMatieres()) {
            somme += noteDe(m) * m.getCoefficient();
            coefs += m.getCoefficient();
        }
        return coefs > 0 ? somme / coefs : 0.0;
    }

    bool valideUE(const UE& ue) const { return moyenneUE(ue) >= 10.0; }

    double moyenneGenerale() const {
        double somme = 0; int credits = 0;
        for (const UE& ue : filiere->getUEs()) {
            somme += moyenneUE(ue) * ue.getCredit();
            credits += ue.getCredit();
        }
        return credits > 0 ? somme / credits : 0.0;
    }

    void afficherReleve() const {
        cout << fixed << setprecision(2);
        cout << "===== RELEVE DE NOTES =====\n"
             << nom << " " << prenom << " (" << matricule << ")\n"
             << "Filiere : " << filiere->getNom() << " - " << niveau << "\n\n";
        for (const UE& ue : filiere->getUEs()) {
            cout << ue.getIntitule() << " [" << ue.getCredit() << " credits]\n";
            for (const Matiere& m : ue.getMatieres())
                cout << "   " << setw(22) << left << m.getIntitule()
                     << " coef " << m.getCoefficient()
                     << "  note : " << noteDe(m) << "\n";
            cout << "   Moyenne UE : " << moyenneUE(ue) << "  -> "
                 << (valideUE(ue) ? "VALIDEE" : "NON VALIDEE") << "\n\n";
        }
        cout << "Moyenne generale : " << moyenneGenerale() << "/20\n";
    }
};

// ---------- main ----------
int main() {
    UE algo("UE1", "Algorithmique", 6);
    algo.ajouterMatiere(Matiere("M1", "Programmation C++", 3));
    algo.ajouterMatiere(Matiere("M2", "Structures de donnees", 2));

    UE maths("UE2", "Mathematiques", 4);
    maths.ajouterMatiere(Matiere("M3", "Analyse", 2));
    maths.ajouterMatiere(Matiere("M4", "Algebre", 2));

```

```
128     Filiere info("MI", "Mathematiques-Informatique");
129     info.ajouterUE(algo);       // la filiere doit etre complete AVANT
130     info.ajouterUE(maths);      // de creer les notes (pointeurs stables)
131
132     Etudiant e("E001", "Doe", "John", "L3", &info);
133     const auto& ues = info.getUEs();
134     e.ajouterNote(ues[0].getMatieres()[0], 14);
135     e.ajouterNote(ues[0].getMatieres()[1], 11);
136     e.ajouterNote(ues[1].getMatieres()[0], 8);
137     e.ajouterNote(ues[1].getMatieres()[1], 10);
138
139     e.afficherReleve();
140     return 0;
141 }
```

## 6 Vérification à la main (jeu d'essai)

**UE1 :**

$$M_{UE1} = \frac{14 \times 3 + 11 \times 2}{3 + 2} = \frac{64}{5}$$

$M_{UE1} = 12{,}80 \geq 10 \Rightarrow$ **UE1 validée**

**UE2 :**

$$M_{UE2} = \frac{8 \times 2 + 10 \times 2}{2 + 2} = \frac{36}{4}$$

$M_{UE2} = 9{,}00 < 10 \Rightarrow$ **UE2 non validée**

**Moyenne générale :**

$$M_G = \frac{12{,}8 \times 6 + 9 \times 4}{6 + 4} = \frac{76{,}8 + 36}{10}$$

$M_G = 11{,}28/20$

C'est exactement ce qu'affiche le programme.

## 7 Notions mobilisées

- **Encapsulation** : attributs `private`, accès par getters et méthodes métier.
- **Composition** : `Filiere` contient `vector<UE>`, `UE` contient `vector<Matiere>` (copies par valeur, durée de vie liée).
- **Agrégation / association** : `Etudiant` référence `Filiere` et `Note` référence `Matiere` par pointeur (durée de vie indépendante).
- **Collections** : `std::vector` d'objets.
- **Multiplicités** : traduites par la présence d'un vecteur (0..∗, 1..∗) ou d'un pointeur unique (1).

**Remarque.** Les pointeurs vers `Matiere` ne sont valides que si les vecteurs ne sont plus modifiés ; c'est pourquoi la filière est construite entièrement avant la saisie des notes.